```
{
 "cells": [
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "# YouTube Video Data and Transcript Extractor\n",
    "\n",
    "This notebook processes a list of YouTube video URLs to extract
key information for each video:\n",
    "\n",
    "- **Video Title**\n",
    "- **Channel Name**\n",
    "- **Upload Date**\n",
    "- **Full Transcript with Timestamps**\n",
    "\n",
    "The collected data for each video is saved as a separate JSON
file in a designated output folder."
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "### Step 1: Install Required Libraries\n",
    "\n",
    "First, we need to install the necessary Python libraries. \n",
    "- `pytube` is used to access video metadata (title, channel,
etc.).\n",
    "- `youtube-transcript-api` is used to fetch the video's
transcript."
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "metadata": {},
   "outputs": [],
   "source": [
    "%pip install pytube youtube-transcript-api"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "### Step 2: Import Libraries\n",
    "\n",
```

```
    "Now, let's import the modules we'll be using throughout the
script."
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "metadata": {},
   "outputs": [],
   "source": [
    "import os\n",
    "import json\n",
    "import re\n",
    "from pytube import YouTube\n",
    "from youtube_transcript_api import YouTubeTranscriptApi,
NoTranscriptFound, TranscriptsDisabled"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "### Step 3: Define the List of YouTube URLs\n",
    "\n",
    "Add the YouTube video URLs you want to process into the
`video_urls` list below. I've included a few examples to get you
started."
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "metadata": {},
   "outputs": [],
   "source": [
    "video_urls = [\n",
    "    'https://www.youtube.com/watch?v=dQw4w9WgXcQ', # A classic
example\n",
    "    'https://www.youtube.com/watch?v=mPZEQg30_w4', # A short
nature video\n",
    "    'https://www.youtube.com/watch?v=roO5B9_v_4g', # A tech
talk\n",
    "    'https://www.youtube.com/watch?v=invalidurl' # Example of a
bad URL\n",
    "]"
   ]
  },
  {
```

```
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "### Step 4: Set Up Output Folder\n",
    "\n",
    "This cell defines the name of the folder where the data files
will be saved and creates it if it doesn't already exist."
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "metadata": {},
   "outputs": [],
   "source": [
    "OUTPUT_FOLDER = 'youtube_video_data'\n",
    "if not os.path.exists(OUTPUT_FOLDER):\n",
    "    os.makedirs(OUTPUT_FOLDER)\n",
    "    print(f\"Created output folder: {OUTPUT_FOLDER}\")\n",
    "else:\n",
    "    print(f\"Output folder '{OUTPUT_FOLDER}' already exists.\")"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "### Step 5: Main Processing Loop\n",
    "\n",
    "This is the core of the notebook. It iterates through each URL,
fetches the data, and saves it to a file.\n",
    "\n",
    "It includes error handling for common issues like invalid URLs or
videos without transcripts."
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "metadata": {},
   "outputs": [],
   "source": [
    "def sanitize_filename(title):\n",
    "    \"\"\"Removes characters that are invalid in
filenames.\"\"\"\n",
    "    # Remove invalid characters\n",
    "    sanitized = re.sub(r'[\\\\/:*?\"<>|]', '_', title)\n",
    "    # Replace spaces with underscores for cleaner filenames\n",
```

```
    "    sanitized = re.sub(r'\\s+', '_', sanitized)\n",
    "    # Truncate to a reasonable length\n",
    "    return sanitized[:100]\n",
    "\n",
    "for url in video_urls:\n",
    "    print(f\"\\nProcessing video: {url}\")\n",
    "    try:\n",
    "        # --- 1. Get Video Metadata using Pytube ---\n",
    "        yt = YouTube(url)\n",
    "        video_id = yt.video_id\n",
    "        video_title = yt.title\n",
    "        channel_name = yt.author\n",
    "        upload_date = yt.publish_date.strftime('%Y-%m-%d') if
yt.publish_date else 'N/A'\n",
    "\n",
    "        print(f\"  > Title: {video_title}\")\n",
    "\n",
    "        # --- 2. Get Transcript using youtube-transcript-api
---\n",
    "        transcript = []\n",
    "        try:\n",
    "            transcript_list =
YouTubeTranscriptApi.get_transcript(video_id)\n",
    "            transcript = transcript_list\n",
    "            print(f\"  > Transcript found with {len(transcript)}
segments.\")\n",
    "        except (NoTranscriptFound, TranscriptsDisabled):\n",
    "            print(\"  > WARNING: No transcript available for this
video.\")\n",
    "            transcript = [{'text': 'No transcript found.',
'start': 0, 'duration': 0}]\n",
    "        \n",
    "        # --- 3. Consolidate Data ---\n",
    "        video_data = {\n",
    "            'url': url,\n",
    "            'title': video_title,\n",
    "            'channel_name': channel_name,\n",
    "            'upload_date': upload_date,\n",
    "            'transcript': transcript\n",
    "        }\n",
    "        \n",
    "        # --- 4. Save to a JSON file ---\n",
    "        filename = sanitize_filename(video_title) + '.json'\n",
    "        filepath = os.path.join(OUTPUT_FOLDER, filename)\n",
    "        \n",
    "        with open(filepath, 'w', encoding='utf-8') as f:\n",
    "            json.dump(video_data, f, ensure_ascii=False,
indent=4)\n",
```

```
    "            \n",
    "        print(f\"  > Successfully saved data to {filepath}\")\n",
    "\n",
    "    except Exception as e:\n",
    "        print(f\"  > ERROR: Could not process video. Reason:
{e}\")\n",
    "\n",
    "print(\"\\n--- All videos processed! ---\")"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "### Finished!\n",
    "\n",
    "Check the `youtube_video_data` folder in the same directory as
this notebook to find your JSON files."
   ]
  }
 ],
 "metadata": {
  "kernelspec": {
   "display_name": "Python 3",
   "language": "python",
   "name": "python3"
  },
  "language_info": {
   "codemirror_mode": {
    "name": "ipython",
    "version": 3
   },
   "file_extension": ".py",
   "mimetype": "text/x-python",
   "name": "python",
   "nbconvert_exporter": "python",
   "pygments_lexer": "ipython3",
   "version": "3.9.7"
  }
 },
 "nbformat": 4,
 "nbformat_minor": 4
}
```